



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

SEZG651/ SSZG653 Software Architectures Module 3

CS05 Architecturally Significant Requirements



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Architecturally Significant Requirements

Harvinder S Jabbal
CSIS, Work Integrated Learning Programs



Software Architecture :

Architecturally Significant Requirements (ASR)

Agenda



- Architecturally Significant Requirements (ASR)
- Identifying ASR from Requirement Documents
- Interviewing Stakeholders (QAW – Quality Attributes Workshop)
- Identifying Business Goals (PALM – Pedigreed Attribute eLicitation Method)
- Building Utility Tree
- What will be the best way forward?

Architecturally Significant Requirements (ASR)



- An architecturally significant requirement (ASR) is a requirement that will have a profound effect on the architecture of a system
- Architecturally significant requirements are those requirements that have a measurable effect on a computer system's architecture.
- This can comprise both software and hardware requirements. They are a subset of requirements, the subset that affects the architecture of a system in measurably identifiable ways.
- Architecture might well be dramatically different in the absence of such a requirement.
- ASRs play a key role in determining the overall structure of a software system and influence design decisions early in the development process
- Important to categorize requirements into Critical, Important and Useful
- Even in critical applications only 40 % of software features are used.



Architecturally Significant Requirements (ASR)

- Indicators for architectural significance include
 - The requirement holds significant business value and/or presents a technical risk.
 - The requirement is important to a key stakeholder with considerable influence.
 - The requirement is unique, meaning that no existing components within the architecture address it.
 - The requirement has quality of service (QoS) or service level agreement (SLA) aspects that differ from those currently accommodated by the evolving architecture.
 - The requirement has previously led to budget overruns or client dissatisfaction in a project with a similar context.
- Common example of ASRs are
 - Scalability: System should support future growth with minimal effort.
 - Security: System must protect data and prevent unauthorized access.
 - Availability: System should have a specific uptime or reliability rate.

Architecturally Significant Requirements (ASR)



–Drivers

- Drivers are the key factors that influence the identification and prioritization of these requirements
 - Business Goals and Constraints (high uptime, horizontal scaling)
 - Stakeholder Concerns (prioritize one requirement over other – security over performance)
 - Quality Attributes (Performance, availability, usability etc.)
 - Regulatory and Compliance Requirements (Ex: GDPR, HIPAA)
 - Technology Constraints and Opportunities (development framework, deployment environment , database etc.)
 - Cost and Budget Constraints
 - Time to Market

Challenging requirements



- Simple functional requirements like ‘System should be able to create an order’, ‘System should keep track of inventory’, etc. are straight forward to implement
- Some functional requirements such as ‘We should be able to detect fraud in real time and block the user’ may not be that straight forward
- Some non-functional requirements like the ‘System should be easy to customize for a different organization” may not be straight forward to implement
- Systems must remain operational even under heavy loads or in case of hardware failure.
- Handling millions of simultaneous user requests without service degradation (e.g., streaming platforms like Netflix or YouTube).
- A healthcare application storing and transmitting electronic health records (EHRs) must be HIPAA compliant and immune to security breaches.
- Such challenging requirements are called ‘Architecturally Significant Requirements’ (ASR)

Examples of ASRs



- Audit trail
 - Alert & Notifications
 - Variety of data input mechanisms
 - Third party interaction
 - Workflow
 - User behaviour analysis (ex. in e-commerce)
 - A user-friendly and responsive web or mobile application
 - Business to maintain their own rules to facilitate decision making.
-
- All such requirements need to be probed in detail, and we need to understand the specific scenarios in order to decide the design approach

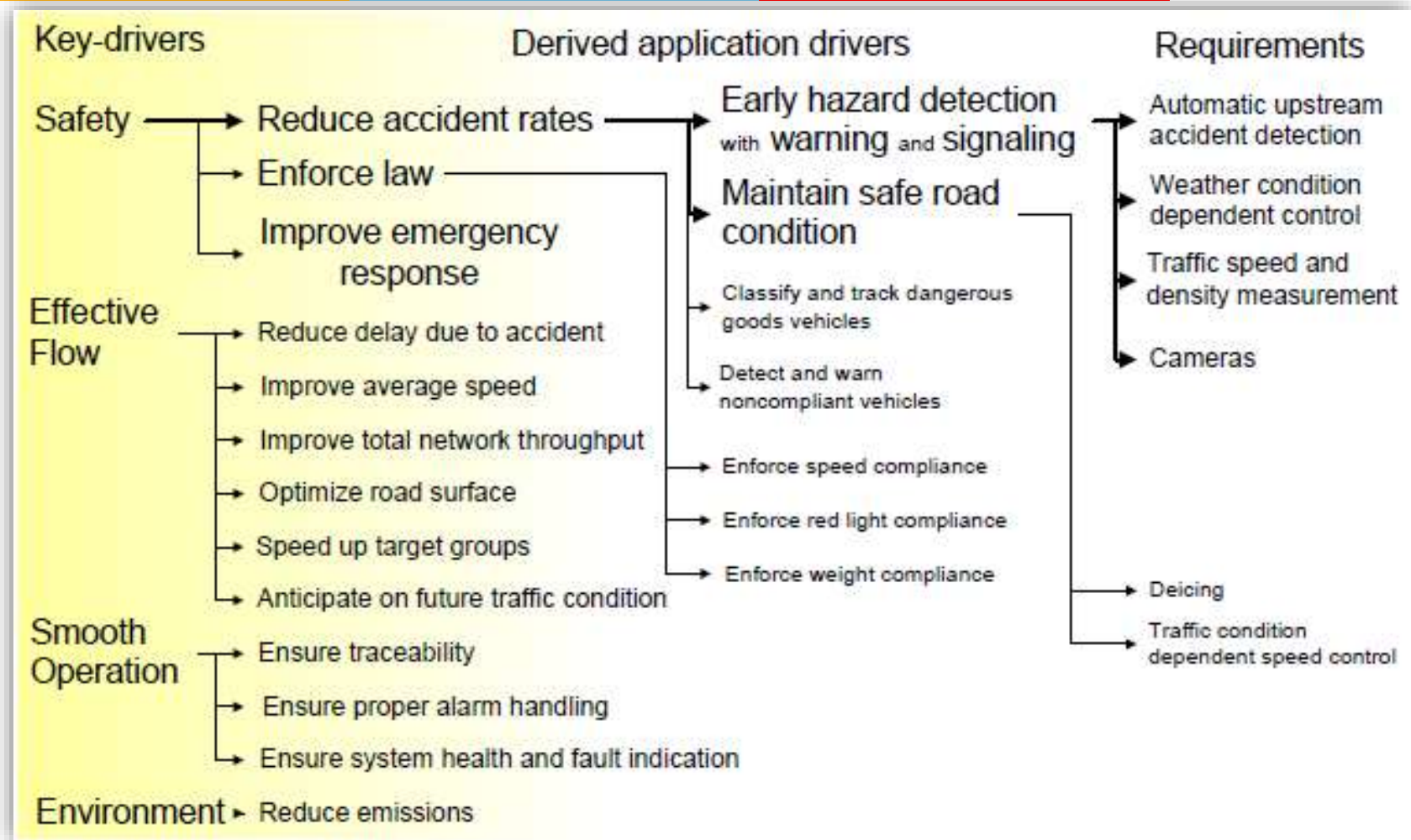
Deeper understanding of requirements helps identify ASRs



Audit trail: Should we use File or DB? This will depend on how the audit trail is going to be used – are we going to perform some kind of analysis of the activities or are we looking for a specific activity

Third party interaction: We need to understand what communication mechanisms are supported by the 3rd party.

Notifications: Should it be an SMS notification or eMail notification or notifying an application via message queue? Should it be a one way notification or do we need an Acknowledgement?





Different ways to identify ASRs

The requirements documents or user stories are an obvious starting point for identifying potential ASR candidates. However, there are additional methods for discovering ASRs

- Requirements
- Design Decisions
- Business goals
- Stakeholder
- Utility Tree



ASR from Requirement Documents

- Do requirements specification follow the principle of “**MoSCoW**” style (**must, should, could, won’t**), or as a collection of “user stories”?
- Do the requirement capturing exercise include Quality Attributes as well in equal proportions as per Software Engineering Body of Knowledge (**SWEBOK**) recommendations?
- Have you come across BSR (Basic Skill Requirements) with explicit requirements on “high usability”, “System Modularity”, “performance through Resilience”, “Fault detection” etc.,?
- So, in the absence of BSR not reflecting the QA clearly, it is a challenge for an ARCHITECT to extract ASR. However, that makes the Role of an Architect requisite.



Look for leads from Design Decision

LOOK FOR LEADS FROM DESIGN DECISION

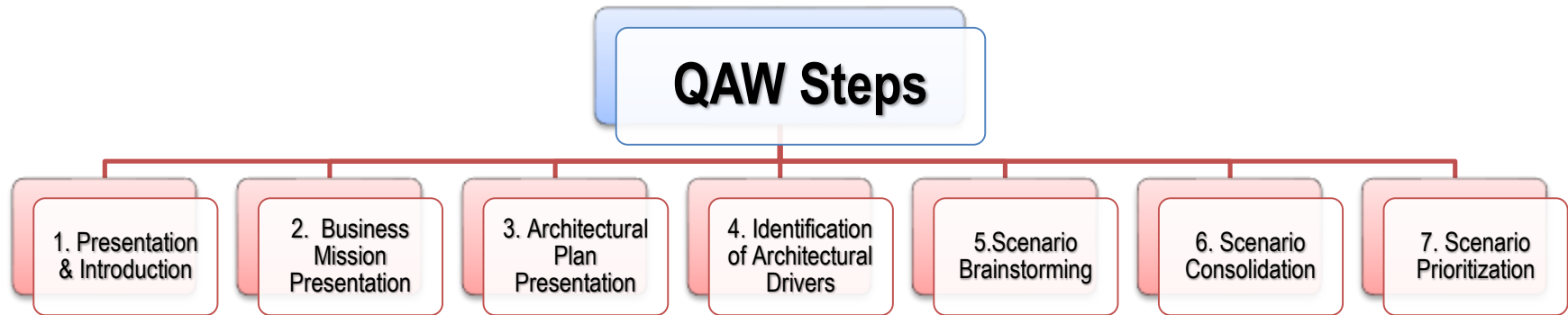
Design Decision Category	Look for Requirements addressing
Allocation of Responsibilities	Planned evolution of responsibilities, user roles, system modes, major processing steps, commercial packages
Coordination Model	Properties of the coordination (timeliness, currency, completeness, correctness, and consistency) Names of external elements, protocols, sensors or actuators (devices), middleware, network configurations (including their security properties) Evolution requirements on the list above
Data Model	Processing steps, information flows, major domain entities, access rights, persistence, evolution requirements
/cont.....	



Look for leads from Design Decision

....cont/	
Data Model	Processing steps, information flows, major domain entities, access rights , persistence, evolution requirements
Management of Resources	Time, concurrency, memory footprint, scheduling, multiple users , multiple activities , devices, energy usage, soft resources (buffers, queues, etc.) Scalability requirements on the list above
Mapping among Architectural Elements	Plans for teaming, processors, families of processors, evolution of processors, network configurations .
Binding Time Decisions	Extension of or flexibility of functionality , regional distinctions, language distinctions, portability, calibrations, configurations
Choice of Technology	Named technologies, changes to technologies (planned and unplanned)

Quality Attributes Workshop (QAW) - Expectation : List of architectural Drivers



1. **Presentation & Introduction** : Everyone introduces themselves, briefly stating their background, their role in the organization, and their relationship to the system being built.
2. **Business Mission Presentation** : The stakeholder representing the business concerns behind the system (typically a manager or management representative) spends about one hour presenting the system's business context, broad functional requirements, constraints, and known quality attribute requirements. The quality attributes that will be refined in later steps

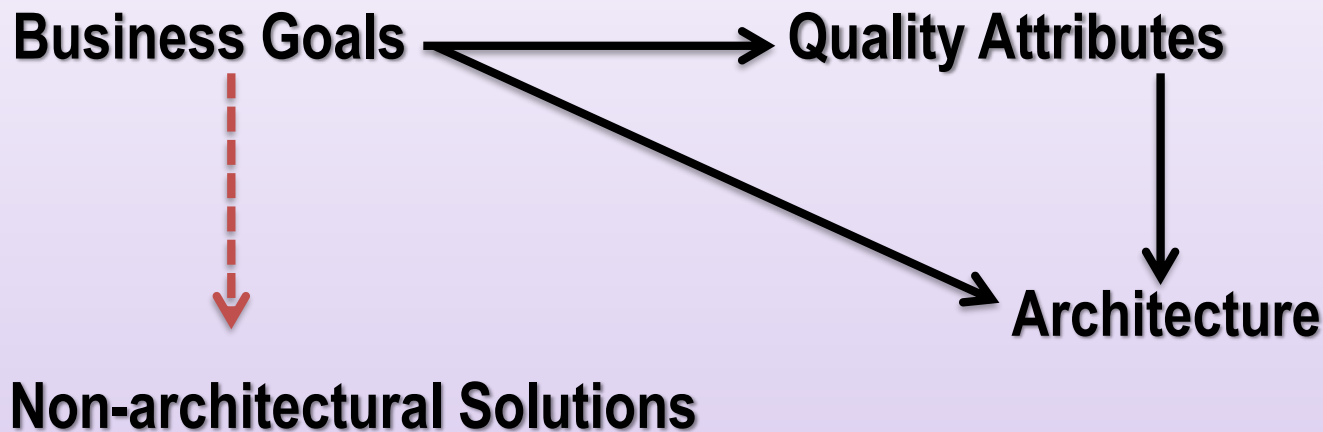


ASR from interviewing Stakeholders

Quality Attributes Workshop (QAW) - Expectation : List of architectural Drivers

3. **Architectural Plan Presentation** : At this point in the workshop, the architect will present the system architectural plans as they stand. This lets stakeholders know the current architectural thinking, to the extent that it exists.
4. **Identification of Architectural Drivers** : The facilitators will share their list of key architectural drivers that they assembled during steps 2 and 3, and ask the stakeholders for clarifications, additions, deletions, and corrections. The idea is to reach a consensus on a distilled list of architectural drivers that includes overall requirements, business drivers, constraints, and quality attributes.
5. **Scenario Brainstorming** : Each stakeholder expresses a scenario representing his or her concerns with respect to the system. Facilitators ensure that each scenario has an explicit stimulus and response. The facilitators ensure that at least one representative scenario exists for each architectural driver listed in step 4.
6. **Scenario Consolidation** : Similar scenarios are consolidated where reasonable. Facilitators ask stakeholders to identify those scenarios that are very similar in content. Scenarios that are similar are merged, as long as the people who proposed them agree and feel that their scenarios will not be diluted in the process
7. **Scenario Prioritization** : Prioritization of the scenarios is accomplished by allocating each stakeholder a number of votes equal to 30 percent of the total number of scenarios generated after consolidation

ASR from Business GOALS



Format : “X” to “Y” by when? : “X “ - Current State : “Y” - Future state

1. Business goals often lead to quality attribute requirements
2. Business goals may directly affect the architecture without precipitating a quality attribute requirement at all
3. No influence at all



ASR from Business GOALS

Format : “X” to “Y” by when? : “X “ - Current State : “Y” - Future state

#	Category	Example
1	Contributing to the <u>growth and continuity</u> of the organization	Market share by 15%
2	Meeting <u>financial</u> objectives	30 % Profit after tax
3	Meeting <u>personal</u> objectives	Learn New Technologies
4	Meeting <u>responsibility to employees</u>	Reduce work load by 8 %
5	Meeting responsibility to <u>society</u>	Green Computing – Less CO2
6	Meeting responsibility to <u>state</u>	Regulatory Compliance
7	Meeting responsibility to <u>shareholders</u>	1:3 Share Dividend
8	Managing <u>market position</u>	IPR - 50 Design Registrations
9	Improving <u>business processes</u>	Productivity Improvement
10	Managing the <u>quality and reputation</u> of products	Automated Testing Strategy
11	Managing change in <u>environmental factors</u>	US Dollar Volatility

19

Example of Business Goal: BHIM

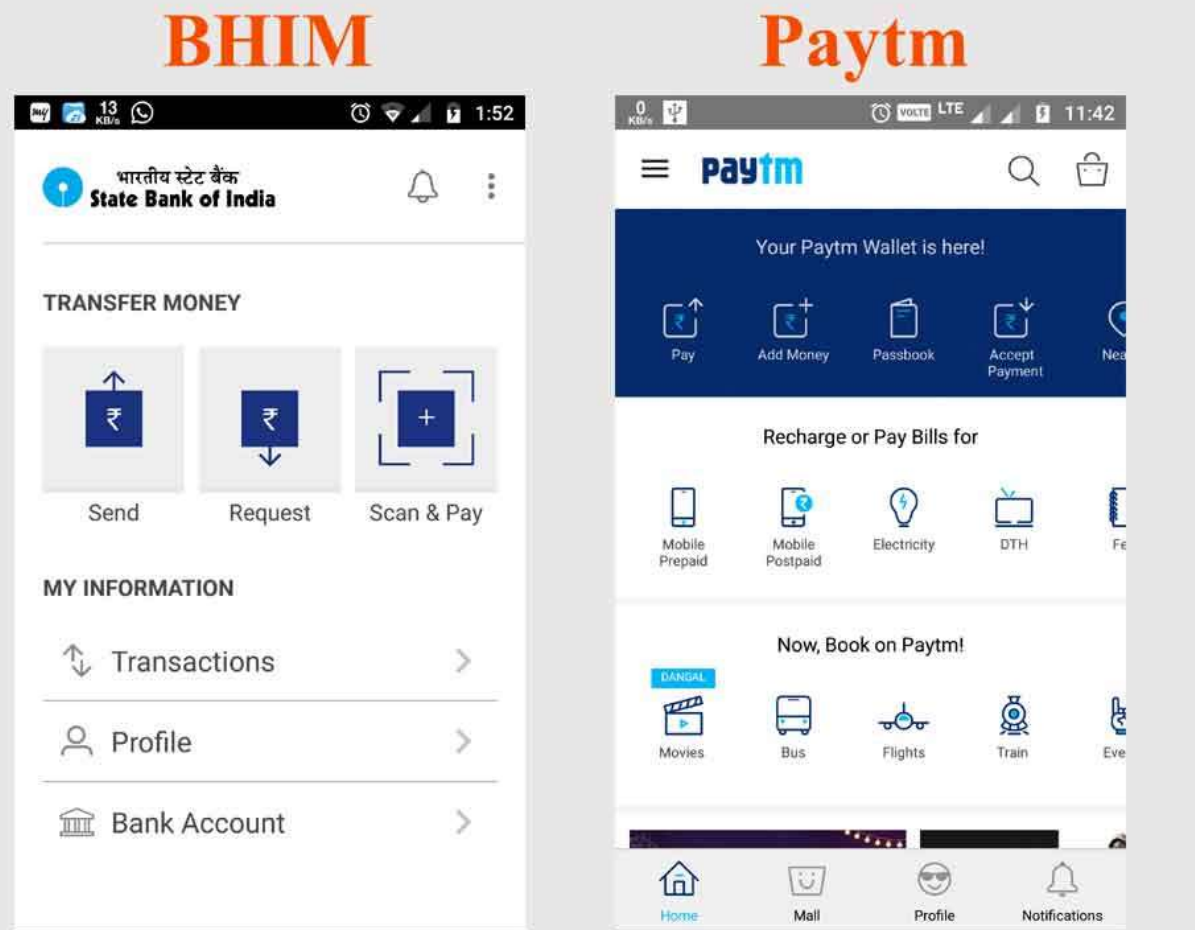


Examples of business goals:

- Govt. wanted to speed-up cashless transactions
- So Government introduce BHIM app

This translates into QAs:

- Usability (easy steps)
- Security (3-factors)



Now send & receive money instantly using your mobile.



Format for Presenting Final Business Goals

2. Goal-subject	...	3. Goal-object	...	5. Goal	...	4. Environment	...	6. Goal-measure (examples, based on goal categories)	7. Value
Any stakeholder or stakeholder group identified as having a <i>legitimate</i> interest in the system	...	Individual	...	Contributing to the growth and continuity of the organization	...	Social (includes political)	...	Time that business remains viable	1-n
	...	System	...	Meeting financial objectives	...	Legal	...	Financial performance vs. objectives	1-10
	...	Portfolio	...	Meeting personal objectives	...	Competitive	...	Promotion or raise achieved in period	H-M-L
	...	Organization's employees	...	Meeting responsibility to employees	...	Customer	...	Employee satisfaction; turnover rate	Resources willing to expend
	...	Organization's shareholders	...	Meeting responsibility to society	...	Technological	...	Contribution to trade deficit/surplus	
	...	Organization	...	Meeting responsibility to state	...		...	Stock price, dividends	
	...	Nation	...	Meeting responsibility to shareholders	...		...	Market share	
	...	Society	...	Managing market position	...		...	Time to carry out a business process	
	...		...	Improving business processes	...		...	Quality measures of products	
	...		...	Managing quality and reputation of products	...		...	Technology-related problems	
	...		...	Managing change in environmental factors	...		...	Time window for achievement	



Capturing Business Goal

“Pedigree” means that the business Goal has a *strong background*:

1. **PALM overview presentation**. Overview of PALM, the problem it solves, its steps, and its expected outcomes.
2. **Business driver’s presentation**. Briefing of business drivers by project management. What are the goals of the customer organization for this system? What are the goals of the development organization?
3. **Architecture driver’s presentation**. Briefing by the architect on the driving business and quality attribute requirements: the ASRs.
4. **Business goals elicitation**. Using the standard business goal categories to guide discussion, capture the set of important business goals as scenarios, consolidate and prioritize them to identify important Goals.
5. **Identification of potential quality attributes from business goals**. For each important business goal scenario, participants describe a quality attribute that would help achieve it
6. **Assignment of pedigree to existing quality attributes drivers**. For each architectural driver named in step 3, we identify which business goals it is there to support. For example: Why is there a 40-millisecond performance requirement? Why not 60 milliseconds? Or 80 milliseconds?
7. **Exercise conclusion**. Review of results, next steps, and participant feedback



ASR in Utility Tree

- An Architecturally Significant Requirement (ASR) in the context of a Utility Tree refers to a critical quality attribute that directly influences the system's architecture.
- The Utility Tree is a hierarchical representation used in software architecture to capture and prioritize key quality attributes, breaking them down into more detailed scenarios.
- The Utility Tree helps architects and stakeholders systematically explore and refine ASRs.
- ASRs Fit into the Utility Tree
 - Root (Utility) -represents the system's overall utility, which is a measure of how well the system satisfies its stakeholders' needs
 - Branches (Quality Attributes)-represent key quality attributes such as performance, security, scalability, availability, and more
 - Leaf Nodes (Quality Scenarios) -Each quality attribute breaks down into more specific scenarios, detailing how the system should behave under certain conditions.



ASR in Utility Tree

Quality Attribute	Attribute Refinement	ASR
Performance	Transaction response time	A user updates a patient's account in response to a change-of-address notification while the system is under peak load, and the transaction completes in less than 0.75 second. (H,M)
		A user updates a patient's account in response to a change-of-address notification while the system is under double the peak load, and the transaction completes in less than 4 seconds. (L,M)
	Throughput	At peak load, the system is able to complete 150 normalized transactions per second. (M,M)
Usability	Proficiency training	A new hire with two or more years' experience in the business becomes proficient in Nightingale's core functions in less than 1 week. (M,L)
		A user in a particular context asks for help, and the system provides help for that context, within 3 seconds. (H,M)
	Normal operations	A hospital payment officer initiates a payment plan for a patient while interacting with that patient and completes the process without the system introducing delays. (M,M)



ASR in Utility Tree

Quality Attribute	Attribute Refinement	ASR
Configurability	User-defined changes	A hospital increases the fee for a particular service. The configuration team makes the change in 1 working day; no source code needs to change. (H,L)
Maintainability	Routine changes	A maintainer encounters search- and response-time deficiencies, fixes the bug, and distributes the bug fix with no more than 3 person-days of effort. (H,M)
		A reporting requirement requires a change to the report generating metadata. Change is made in 4 person-hours of effort. (M,L)
	Upgrades to commercial components	The database vendor releases a new version that must be installed in less than 3 person-weeks. (H,M)
Extensibility	Adding new product	A product that tracks blood bank donors is created within 2 person-months. (M,M)

ASR in Utility Tree

Quality Attribute	Attribute Refinement	ASR
Security	Confidentiality	A physical therapist is allowed to see that part of a patient's record dealing with orthopaedic treatment but not other parts nor any financial information. (H,M)
	Integrity	The system resists unauthorized intrusion and reports the intrusion attempt to authorities within 90 seconds. (H,M)
Availability	No Downtime	The database vendor releases new software, which is hot-swapped into place, with no downtime. (H,L)
		The system supports 24/7 web-based account access by patients. (L,L)

H - High Priority

M - Moderate Priority

L - Low Priority

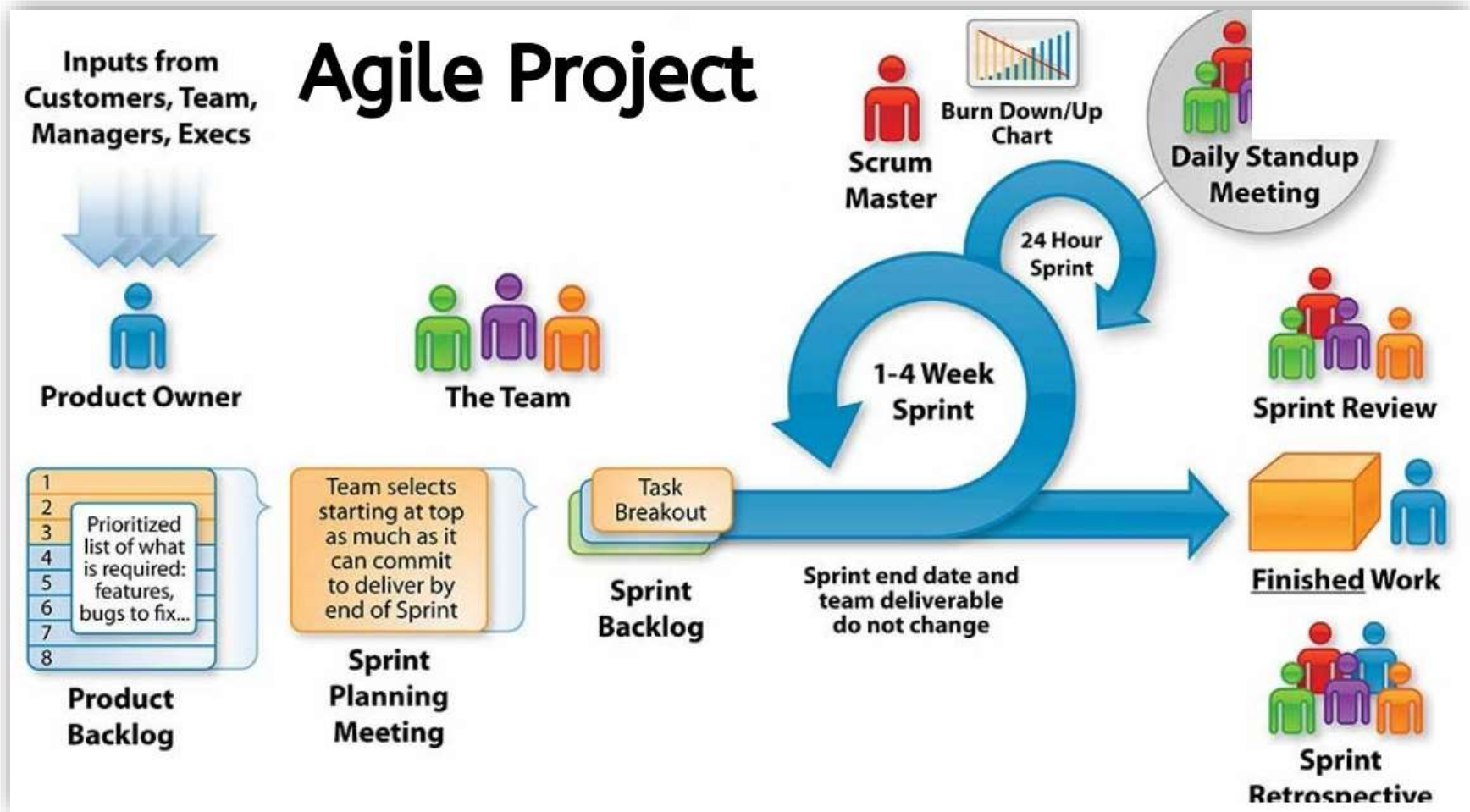
What is the Best Approach?



- How much is the time scheduled in your project for this activity?
- As an architect, even when no BSR, you are accountable for ASR...
- Interview with stakeholders must be planned by you well in advance.....
- Utility Tree is more like a repository approach.....
- ... Choose the one the most suits with the time to fill



Architecture in Agile Projects



Agile Vs. Traditional



Agile Development

Traditional Development

Individuals and interactions

Processes and tools

Working software

Comprehensive documentation

Customer collaboration

Contract negotiation

Responding to change

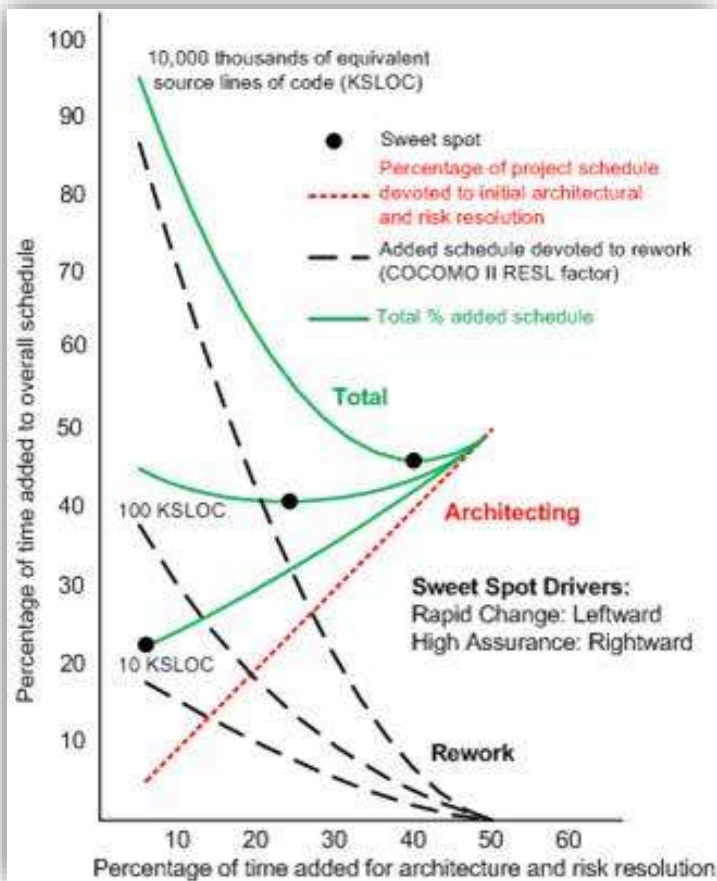
Following a plan

Agile Manifesto :12 Principles



#	Principles	#	Principles
1	Early and continuous delivery of valuable software	7	Working software is the primary measure of progress
2	Welcome changing requirements , even late in development	8	Agile processes promote sustainable development
3	Deliver working software frequently , from a couple of weeks to a couple of months	9	Continuous attention to technical excellence and good design enhances agility.
4	Business people and developers must work together	10	Simplicity is essential
5	Build projects around motivated individuals	11	The best architectures, requirements, and designs emerge from self-organizing teams
6	Face-to-Face conversation	12	At regular intervals, the team reflects on how to become more effective , then tunes and adjusts its behaviour accordingly

Analytics : Up-front Work Vs. Agility

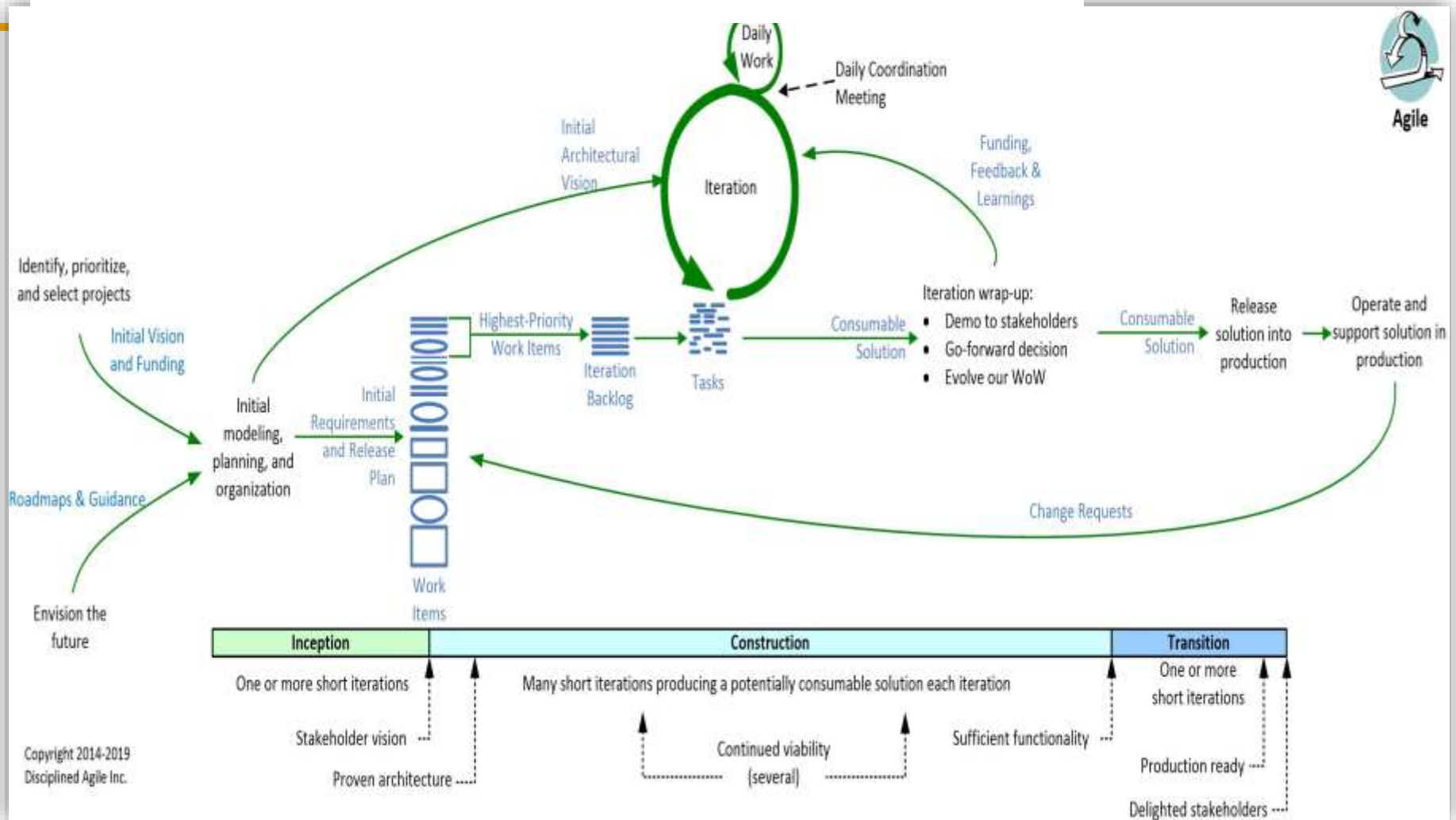


Interpretations:

1. There is one line representing each of these three projects, starting near the Y axis and descending, at different rates, to the X axis at the 50 mark. This shows that adding time for up-front work reduces later rework.
2. when you sum each of those downward-trending lines (for the 10, 100, and 1,000 KSLOC projects) with the upward sloping line for the up-front (initial architecture and risk resolution) work, you get the second set of three lines, which start at the Y axis and meet the upward sloping line at the 50 mark on the X axis.
3. These lines show that there is a sweet spot for each project. For the 10 KSLOC project, the sweet spot is at the far left. This says that devoting much, if any, time to up-front work is a waste for a small project.
4. For the 100 KSLOC project, the sweet spot is at around 20 percent of the project schedule. And for the 1,000 KSLOC project, the sweet spot is at around 40 percent of the project schedule.

A project with a million lines of code is enormously complex, and it is difficult to imagine how Agile principles alone can cope with this complexity if there is no architecture to guide and organize the effort.

BDUF (Big Design Up-front Model) for Agile



Define Architecture completely Before Implementation

August 22, 2026

SSZG653 Software Architectures

Thank you!



[Sat Arch Episode Ep - 23: Architecturally Significant Requirements Part 1 \(youtube.com\)](#)

Assignment work – [10 marks]-

[How To Write A Case Study? | Amazon Case Study Example \(youtube.com\)](#)